

Computer Organization, Spring 2026

Project Assignment 3:

Pipelined Processor CPU Design

Lecturer: 陳雅淑教授

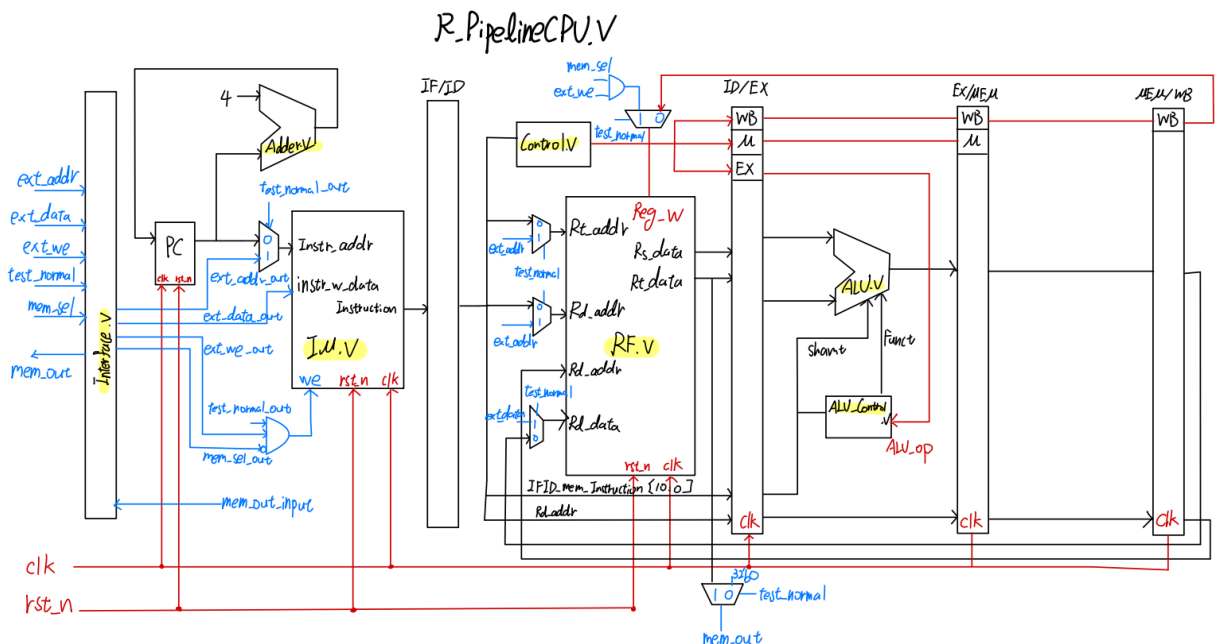
Department: Electrical Engineering

Student ID: B11232009

Name: 胡守曾

R_PipelineCPU

Hardware Architecture



Design Process

這個部分實作一個支援 R_Format 的 32 bit pipeline CPU, 處理器包含 IF、ID、EX、MEM、WB 五個階段, 並透過 IF/ID、ID/EX、EX/MEM、MEM/WB pipeline registers 傳遞資料與控制訊號。

- 各階段執行動作

1. IF

PC 會提供位址給 Instruction Memory 讀取指令。正常執行時, PC 每個 clock 增加 4; 在 test mode 時, 則由外部的 ext_addr 和 ext_data 載入 instruction。而輸出的 Instruction 會被存入 IF/ID register 中。

2. ID

這個部分會將存於暫存器中的指令取出, 並分解送入 Register File 中, 並將 Rs_data, Rt_data, Instruction[10:0] (Shamt, Funct_ctrl) 存入 ID/EX register 中, 同時需要將 Rd_addr 存入, 因為在最後 WB 時需要使用, 避免被覆蓋。而 Control 的部分則是只會使用 Reg_w, ALU_op 這兩個訊號, 也需要將其存入 ID/EX register 中。

3. EX

在 EX 階段, ALU_Control 會透暫存器中的 Funct_ctrl 來送出對應 Funct 給 ALU, 而 ALU 則會依照 ALU Control 的輸出執行對應運算。最終將運算結果存入 EX/MEM register 中, 而需要回傳的 Rd_addr 地址以及 Control signal WB 中的 Reg_w 則依序傳遞。

4. MEM

由於此部分只包含 R_type instruction, 因此不需要使用 Data memory, 不過還是保留五階段的 pipeline 架構, 訊號則會依序傳遞。

5. WB

ALU result 會在這個階段回傳給 RF, 透過 MEM/WB register 中的 WB Reg_w 以及 Rd_addr 可以使其傳回 RF 中。

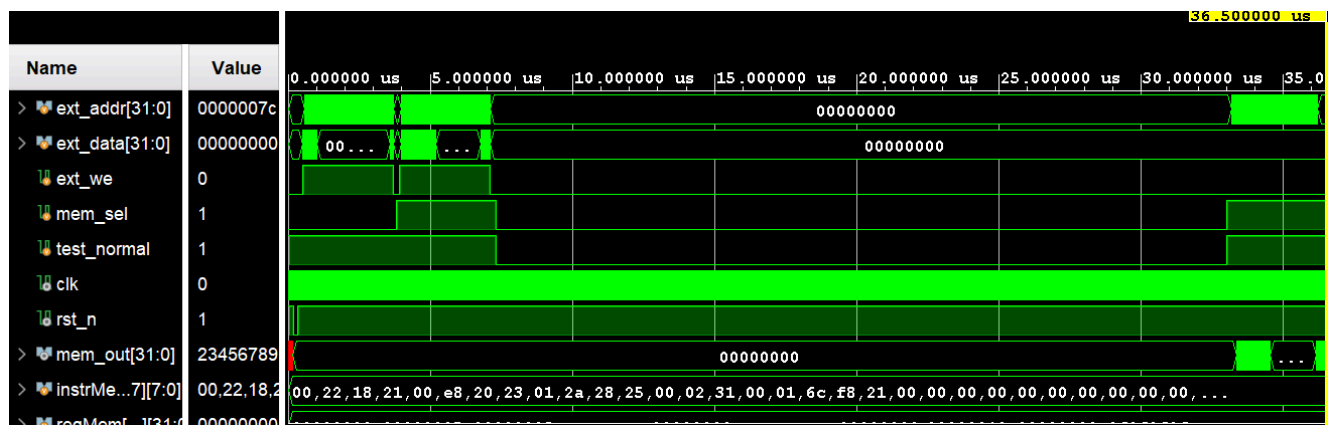
● Datapath & Controlpath

Datapath 從 PC 開始, PC 提供 instruction address 給 Instruction Memory。取出的 instruction 會進入 IF/ID register, 接著在 ID stage 讀取 Register File。讀出的 Rs_data、Rt_data、目的暫存器位址與控制訊號會一起存入 ID/EX register。

Control path 主要由 Control module 和 ALU Control module 組成。由於只支援 R-format, 所以 Control module 會產生固定的 R-format 控制訊號, 例如 RegWrite 啟用、目的暫存器選擇 rd。ALU Control 則根據 funct 欄位決定 ALU 要執行 addu、subu、or 或 sll。

Behavioral Simulation

1. hexadecimal waveforms



2. Vivado tcl console

```

run all
IM loaded
RF Loading...
RF loaded

--- Start to Run Instructions in IM ---
--- Instructions in IM are done ---

--- [Register File Results] ---
RF[ 0] : 00000000
RF[ 1] : 00000005
RF[ 2] : 00000003
RF[ 3] : 00000008
RF[ 4] : ffffffff0
RF[ 5] : ffff0f0f
RF[ 6] : 00000030
RF[ 7] : 00000010

RF[27] : 00000000
RF[28] : 00000000
RF[29] : aaaaaaaa
RF[30] : bbbbbbbb
RF[31] : 23456789

Simulation Completed. By IEESLab

  /\_/\   /\_/\   /\_/\   Congratulations !
( o.o ) ( -.- ) ( ^.^ ) Please Check <<RF_test.out>>
> ^<    > ^<    > ^<    Wishing you continued success and happiness. By IEESLab

$finish called at time : 36500 ns : File "C:/Vivados_Verilog_project/PA3/R_PipelineCP

```

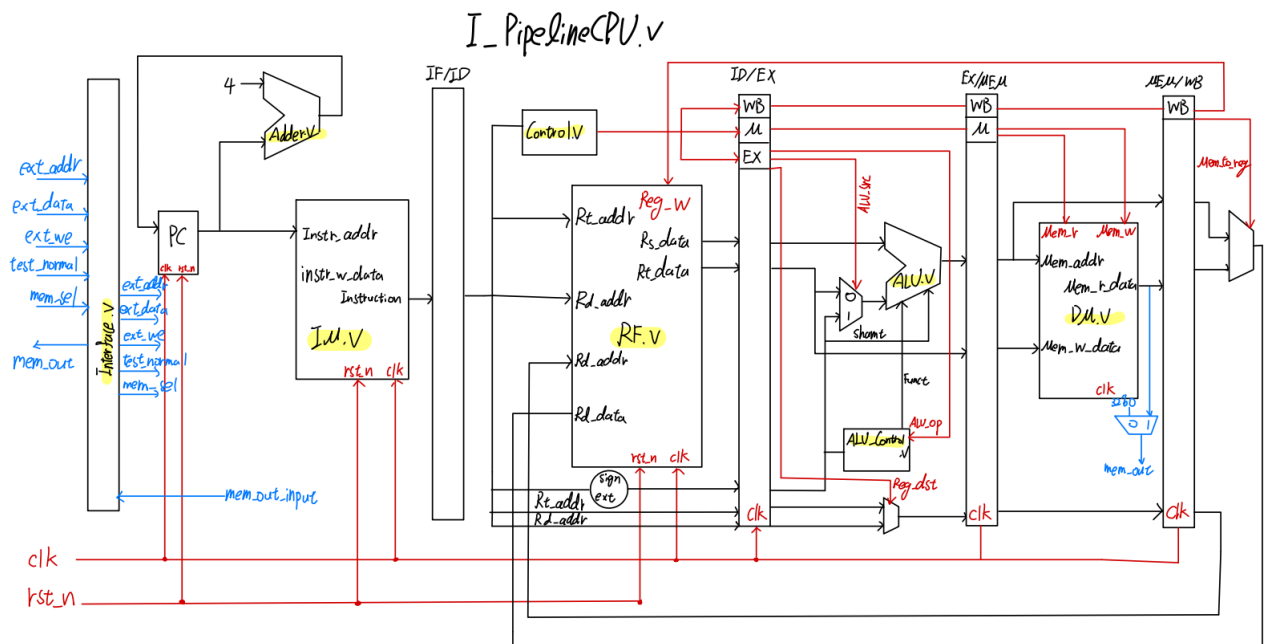
3. Describe

可以透過波形圖看到整個流程分為四個部分: initialization / data loading、instruction execution、result dumping, 以及 simulation finish。前段在 test mode 下透過 ext_addr、ext_data 與 ext_we 載入 IM.dat 以及 RF.dat / DM.dat。接著 test_normal 拉低後進入 normal mode, CPU 開始執行 Instruction Memory 中的指令, 並依序經過 IF、ID、EX、MEM、WB 五個 pipeline stages。由於 testbench 會固定讓 CPU 執行 RUN_CLOCK_CYCLE (256) 個 clock, 以確保所有指令完整通過 pipeline, 因此 execution phase 在 waveform 中會維持較長時間。最後 test_normal 再次拉高, testbench 進入 dump 階段, 透過 mem_out 讀出 RF 或 DM 的結果並進行驗證。

可以透過最後 Vivado tcl console 確認最終結果正確。

I_PipelineCPU

Hardware Architecture



Design Process

這個部分做的是支援 R_format 和 I_format instruction 的 32 bit CPU。由於多了 lw, sw 因此架構上新增了 Data Memory，也需要加入 immediate extension、ALUSrc mux。並且中間暫存器部分也需要存入更多資料，才能正確的傳輸資料。

- 各階段執行動作

1. IF

與 R_PipelineCPU 一樣，PC 會提供位址給 Instruction Memory 讀取指令。正常執行時，PC 每個 clock 增加 4。

2. ID

這個部分會將存於暫存器中的指令取出，並分解送入 Register File 中，在這個部分需要額外多存入 Rt_addr (I-format 回傳資料到 Rt 中)，並且需要先做 sign extension，並且只有 ori 使用 zero extension，最後存入 ID/EX reg 中。

Control 部分對於 DM 額外多增加了 Mem_r 和 Mem_w，並且針對 EX 部分額外加上了 ALU_src, Reg_dst 以及原本就有的 ALU_op。

3. EX

在 EX 階段，ALU Control 會根據 ALUOp 和 funct 決定 ALU 要執行功能，若是 I-format 指令，ALU Control 則根據 ALUOp 直接決定執行加法或 OR。同時使用 ALUSrc mux 決定 ALU 第二個輸入來源。當指令是 R-format 時，ALU 的第二個輸入為 Rt_data；當指令是 I-format 時，ALU 的第二個輸入為 extended immediate。

此外，目的暫存器也會透過 RegDst mux 選擇。R-format 指令寫回 rd, I-format 中的 addiu、lw、ori 則寫回 rt。最後，ALU 運算結果、Rt_data、目的暫存器位址和控制訊號會被存入 EX/MEM register。

4. MEM

在 MEM 階段，只有 I_format 會使用到。lw 會根據 ALU 算出的 address 從 Data Memory 讀出資料；sw 則會將 Rt_data 寫入 Data Memory 對應位址。DM 會透過 pipeline registers 中訊號來控制讀寫。在最後則會將 ALU_result 以及 Mem_r_data (DM 讀出的資料) 存入 MEM/WB register 中。

5. WB

在 WB 階段，會透過 Mem_to_reg 來決定寫回 Register File 的資料來源，如果是 R-format、addiu 或 ori，寫回的資料來自 ALU result；如果是 lw，寫回的資料來自 Data Memory 的 read data。

● Datapath & Controlpath

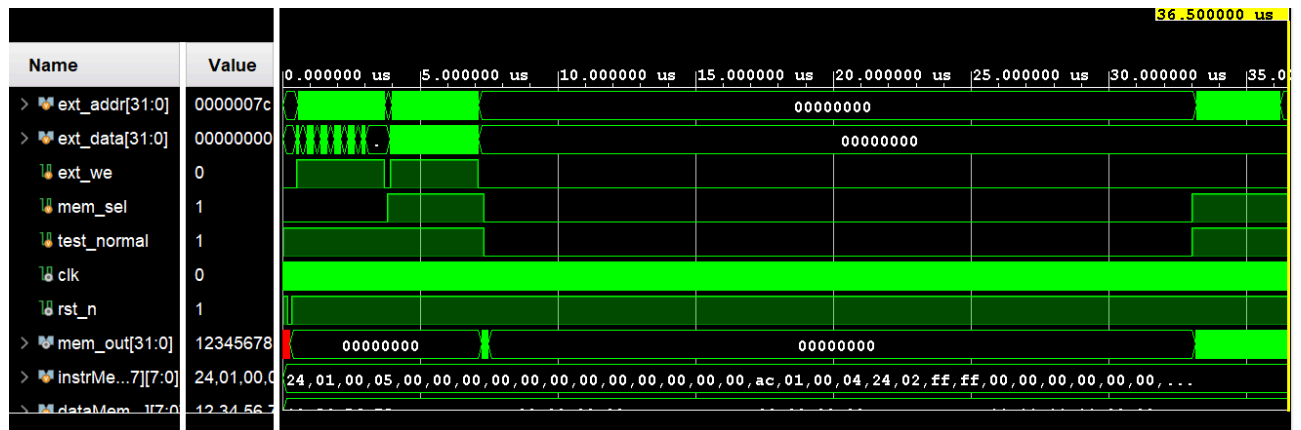
Datapath 從 PC 開始，PC 提供 instruction address 給 Instruction Memory。取出的 instruction 會進入 IF/ID register，接著在 ID stage 進行 decode 並讀取 Register File。Rs_data、Rt_data、extended immediate、目的暫存器位址，以及控制訊號會一起進入 ID/EX register。

在 EX stage，ALU 會根據控制訊號選擇輸入資料。R-format 指令使用兩個 register data 進行運算；I-format 指令則使用 Rs_data 和 extended immediate 進行運算。ALU result 之後會進入 EX/MEM register。若指令是 lw 或 sw，ALU result 會作為 Data Memory address 使用。

Control path 主要由 Control module 和 ALU Control module 組成。Control module 根據 opcode 判斷指令類型，並產生 RegWrite、MemRead、MemWrite、MemtoReg、ALUSrc、RegDst 和 ALUOp 等控制訊號。ALU Control 則根據 ALUOp 和 funct 決定 ALU 實際執行的功能。這些 control signals 會隨著 instruction 經過 pipeline registers 一路傳遞，確保每條 instruction 在正確階段執行正確操作。

Behavioral Simulation

1. hexadecimal waveforms



2. Vivado tcl console

```

run all
IM loaded
DM Loading...
DM loaded

--- Start to Run Instructions in IM ---
--- Instructions in IM are done ---

--- [Data Memory Results] ---
DM[ 0] : 12345678
DM[ 4] : 00000005
DM[ 8] : ffffffff
DM[12] : 000000ff
DM[16] : 12345678
DM[20] : 0000ffff
DM[24] : 11111111
DM[28] : 22222222

DM[112] : 80808080
DM[116] : 90909090
DM[120] : a0a0a0a0
DM[124] : 12345678

Simulation Completed. By IEESLab

    /\_/\    /\_/\    /\_/\    Congratulations !
  ( o.o )  ( -.- )  ( ^.^ )    Please Check <<DM_test.out>>
   > ^ <   > ^ <   > ^ <     Wishing you continued success and happiness.   By IEESLab

$finish called at time : 36500 ns : File "C:/Vivados Verilog project/PA3/I PipelineC

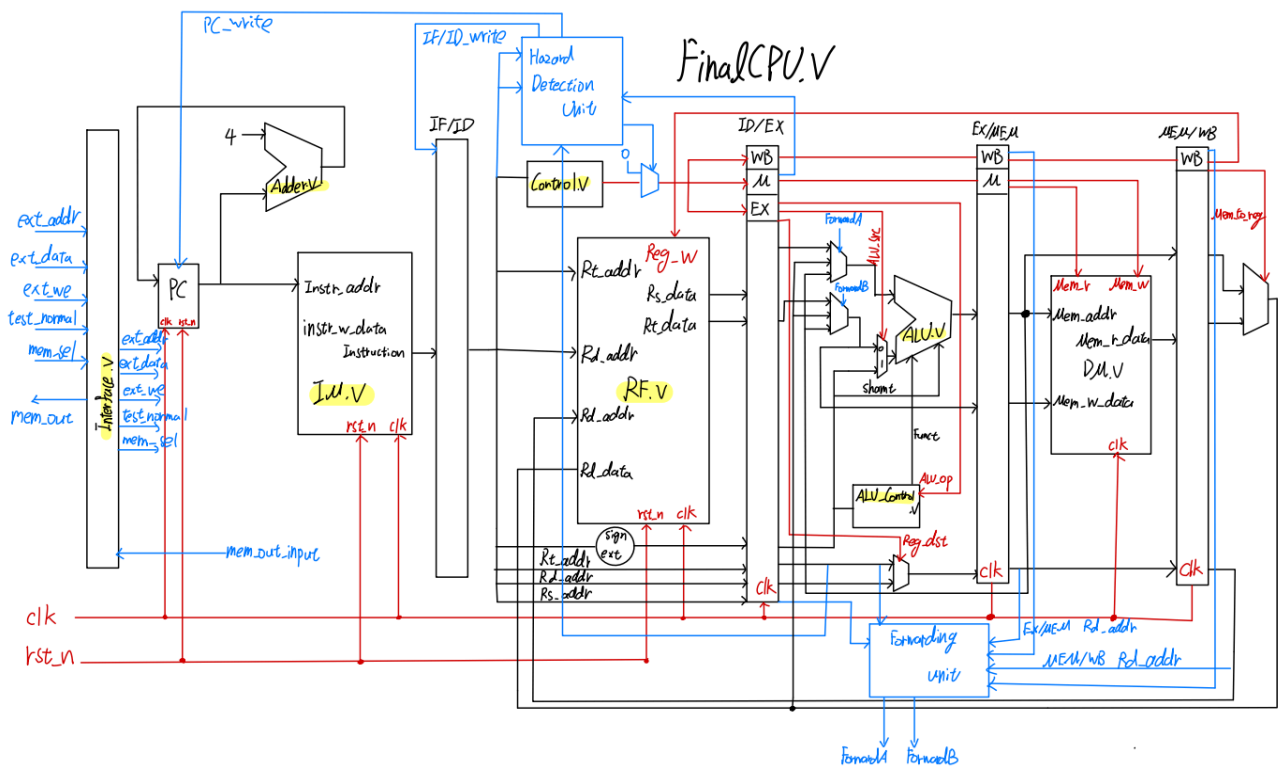
```

3. Describe

由於這個部分沒有做 hazard detection 和 forwarding unit, Instruction 中會有許多 NOP 來避免資料相依性的問題, 可以透過波形圖看到輸出一樣分為四個部分, 不過改成 load data to Data Memory。透過 tcl console 也可以看到 DM 資料都符合預期, 並且由於 DM 是 $128 \times 8\text{-bit}$, 因此顯示部分會是每次加4。

FinalCPU

Hardware Architecture



Design Process

這個部分相較於 I_PipelineCPU 主要差異在於需要處理 pipeline 中的 data hazard。因此架構上額外加入 Forwarding Unit 與 Hazard Detection Unit。Forwarding Unit 用來解決一般 R-format 或 I-format instruction 之間的資料相依問題；Hazard Detection Unit 則主要用來處理 lw-use hazard，當 lw 後面的指令馬上使用 load 的結果時，需要暫停 PC 與 IF/ID register，並在 ID/EX 階段插入 bubble。

- 各階段執行動作

1. IF

與前兩部分大致相同，不過PC 額外受到 PC_write 控制。正常情況下，PC_write = 1，PC 會正常更新；當 Hazard Detection Unit 偵測到 lw-use hazard 時，PC_write = 0，PC 會保持原值，避免下一條 instruction 被錯誤抓取。

2. ID

在 ID 階段新增了 Hazard Detection Unit，當偵測到 lw-use hazard 時，control mux 會將 control signals 清為 0，使 ID/EX 階段插入一個 bubble，停止寫入 IF/ID register，並且暫停 PC + 4，等於做 bubble。並且多存入了 Rs_addr，目的是給下一階段的 Forwarding unit 使用。

3. EX

在 EX 階段，此階段會使用 ALUSrc mux 決定 ALU 第二個輸入來源。R-format 指令的第二個輸入為 Rt_data，而 I-format 指令的第二個輸入為 extended immediate。

Part 3 額外加入 Forwarding Unit，其主要目的是可以提前將資料給下一階段做動作，而不是等到 WB 後才能使用，若前面 instruction 的結果尚未寫回 RF，但目前 instruction 已經需要使用該資料，Forwarding Unit 會透過 ForwardA 和 ForwardB 選擇正確資料來源，避免使用到舊的 RF value。分別會偵測 EX hazard, MEM hazard, 透過判斷 Reg_w, 寫回地址與寫入地址是否有衝突來判斷是否需要 Forward。

並且此階段會透過 Reg_dst 來判斷最後需要寫回的地址。

4. MEM

在 MEM 階段，Data Memory 的讀寫會根據 EX/MEM register 中保存的 MemRead 與 MemWrite 控制訊號執行。需要的資料都會存於 EX/MEM register 中。

5. WB

在 WB 階段，CPU 會根據 MemtoReg 決定寫回 Register File 的資料來源。這個階段的 write-back data 同時也會作為 MEM/WB forwarding 的資料來源，讓後續 instruction 在 EX 階段可以直接使用尚未正式寫回 RF 的結果。

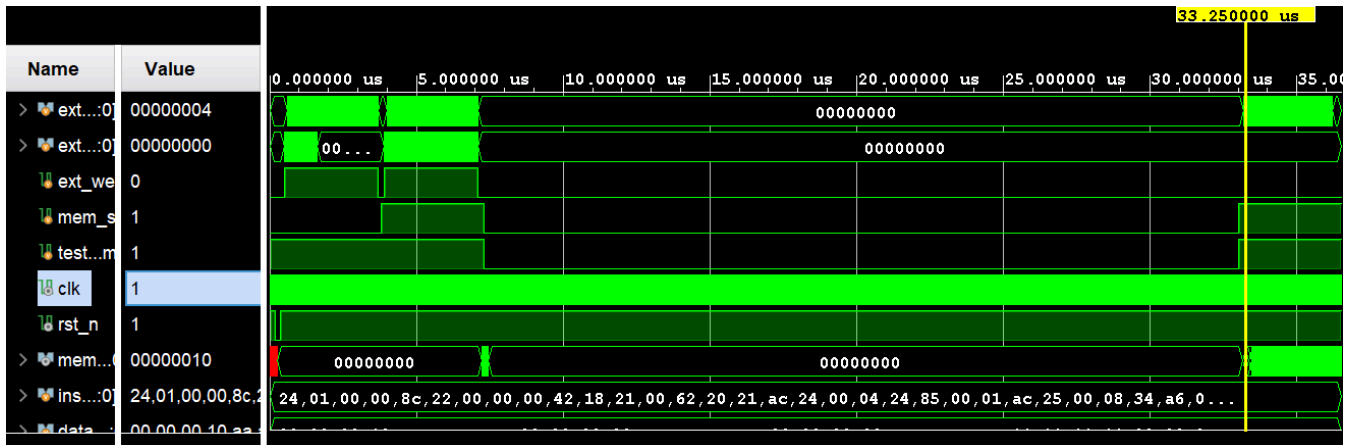
● Datapath & Controlpath

Datapath 由 PC 提供地址至 Instruction Memory 取出指令，存入 IF/ID 暫存器。在 ID 階段進行解碼並讀取 Register File，隨後將暫存器資料、立即數、相關位址與控制訊號鎖存至 ID/EX 暫存器。進入 EX 階段後，ALU 根據 ALUSrc 控制多工器 (MUX) 選擇輸入：R-format 運算兩個暫存器資料，I-format 則運算 Rs_data 與立即數。運算結果 (ALU result) 存入 EX/MEM 暫存器，若為 lw 或 sw 指令，此結果將作為 Data Memory 的存取地址。

Control path 包含 Control、ALU Control、Forwarding 與 Hazard Detection 等單元。Control 模組根據 opcode 產生 RegWrite、MemRead、MemWrite、MemtoReg、ALUSrc、RegDst 與 ALUOp 等訊號；ALU Control 再結合 funct 決定 ALU 運算功能。Forwarding Unit 比較 pipeline 暫存器中的暫存器位址，產生 ForwardA/B 控制 MUX 進行前推。Hazard Detection Unit 則偵測 lw-use hazard，透過 PC_write、IF_ID_write 與 Control_stall 執行 pipeline stall 並插入 bubble。

Behavioral Simulation

1. hexadecimal waveforms



2. Vivado tcl console

```
run all
IM loaded
DM Loading...
DM loaded

--- Start to Run Instructions in IM ---
--- Instructions in IM are done ---

--- [Data Memory Results] ---
DM[ 0] : 00000010
DM[ 4] : 00000030
DM[ 8] : 00000031
DM[12] : 000000ff
DM[16] : 00000ff0
DM[20] : eeeeeeee
DM[24] : 11111111
DM[28] : 22222222

DM[112] : 80808080
DM[116] : 90909090
DM[120] : a0a0a0a0
DM[124] : 12345678

Simulation Completed. By IEESLab

  /\_/\  /\_/\  /\_/\  Congratulations !
( o.o ) ( -.- ) ( ^.^ ) Please Check <<DM_test.out>>
> ^<    > ^<    > ^<    Wishing you continued success and happiness. By IEESLab

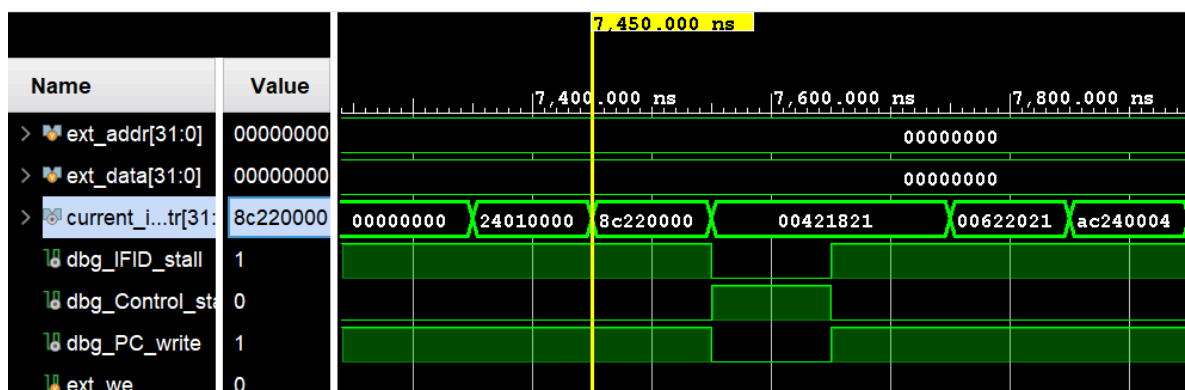
$finish called at time : 36500 ns : File "C:/Vivados_Verilog_project/PA3/FinalCPU/TES
```

3. Describe

```
1 24 01 00 00 // addiu $1, $0, 0
2 8c 22 00 00 // lw    $2, 0($1)
3 00 42 18 21 // addu  $3, $2, $2
4 00 62 20 21 // addu  $4, $3, $2
5 ac 24 00 04 // sw    $4, 4($1)
6 24 85 00 01 // addiu $5, $4, 1
7 ac 25 00 08 // sw    $5, 8($1)
8 34 a6 00 ff // ori   $6, $5, 00ff
9 ac 26 00 0c // sw    $6, 12($1)
10 00 06 39 00 // sll  $7, $6, 4
11 ac 27 00 10 // sw    $7, 16($1)
```

此為 IM 內資料

可以透過 Vivado tcl console 看到最終 DM 結果符合預期, hazards 可以被正確偵測並且做出 forwarding。透過 waveform 則可以看到整體為四個部分, 皆可以正確執行。



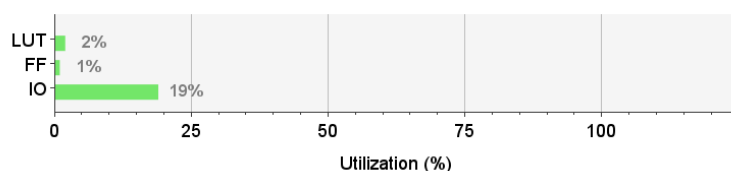
以第二以及第三指令作為範例, 透過加入的testbench可以觀察到 load-use data hazard。由於 lw 指令需要到 MEM stage 才能取得 Data Memory 的資料, 而下一條 addu 指令立即使用 \$2, 因此 forwarding 仍不足以解決此相依性, 必須插入一個 stall。

在 waveform 中, 當 hazard 發生時, dbg_PC_write 會變為 0, 使 PC 暫停更新; dbg_IFID_stall 會變為 1, 使 IF/ID register 保持目前的 addu 指令; dbg_Control_stall 則會變為 1, 將 ID/EX control signal 清為 0 並插入 bubble。由此可確認 hazard detection unit 有正確偵測 load-use hazard, 並使 pipeline stall 一個 clock cycle。

Synthesis Result

hardware utilization

Resource	Utilization	Available	Utilization %
LUT	2475	134600	1.84
FF	3385	269200	1.26
IO	74	400	18.50



timing report

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 43.491 ns	Worst Hold Slack (WHS): 0.134 ns	Worst Pulse Width Slack (WPWS): 49.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 9875	Total Number of Endpoints: 9875	Total Number of Endpoints: 3386
All user specified timing constraints are met.		

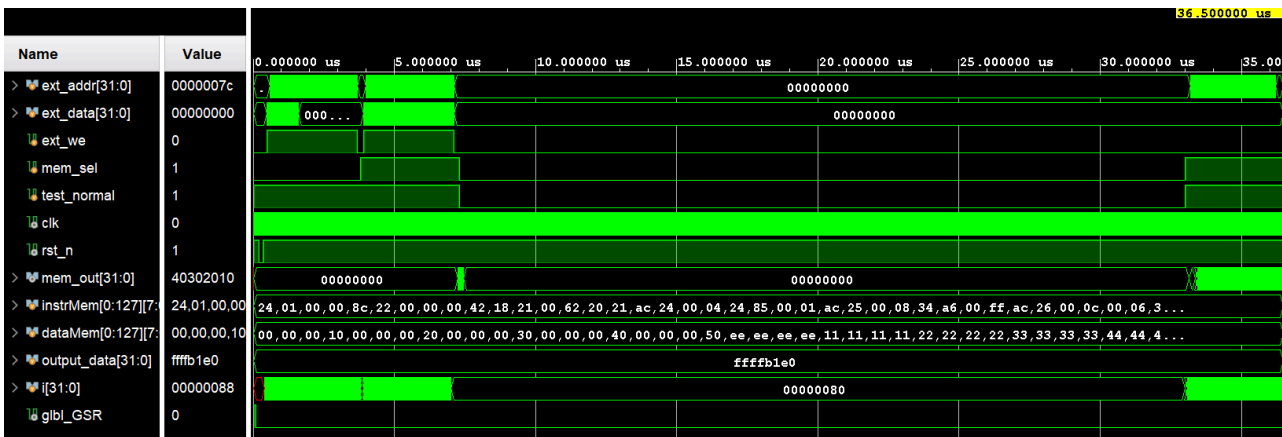
Describe

Synthesis 結果顯示, 本設計已成功通過 timing constraint, Setup、Hold 與 Pulse Width 皆無 violation, 代表電路在目前時脈限制下仍有足夠 timing margin。

硬體使用方面也符合規範。

Post-Synthesis Timing Simulation

hexadecimal waveforms



Vivado tcl console

```
run all
IM loaded
DM Loading...
DM loaded

--- Start to Run Instructions in IM ---
--- Instructions in IM are done ---

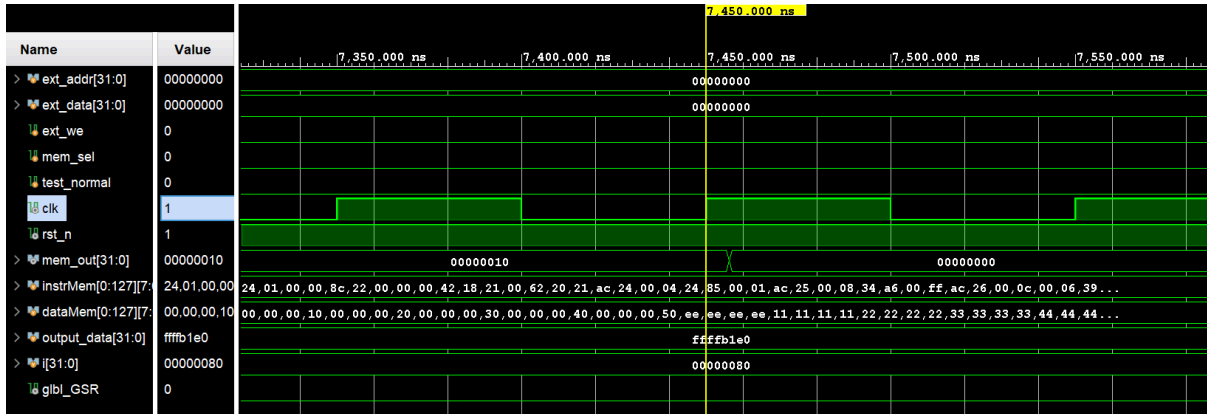
--- [Data Memory Results] ---
DM[ 0] : 00000010
DM[ 4] : 00000030
DM[ 8] : 00000031
DM[12] : 000000ff
DM[16] : 00000ff0
DM[20] : eeeeeeee
DM[24] : 11111111
DM[28] : 22222222
```

Simulation Completed. By IEESLab

```
  /\_/\  /\_/\  /\_/\  Congratulations !
( o.o ) ( -.- ) ( ^_^ ) Please Check <<DM_test.out>>
> ^ <   > ^ <   > ^ <   Wishing you continued success and happiness.  By IEESLab
```

\$finish called at time : 36500 ns : File "C:/Vivados_Verilog_project/PA3/FinalCPU/TESTBED/tb_FinalCPU.v" Line 205
run: Time (s): cpu = 00:00:01 ; elapsed = 00:00:06 . Memory (MB): peak = 3288.910 ; gain = 0.000

Describe



透過圖片可以看到post-synthesis Simulation 會使輸出有些微延遲, clk 上升緣之後, 相關輸出訊號會稍微延後才改變, 這是因為資料需要經過暫存器的 clock-to-Q delay、combinational logic delay, 以及 mux / memory output delay。

FinalCPU Hazard detection unit Testbench

testbench

定義 function

```
fake_instr = 32'b0;
fake_instr[25:21] = in_ifid_rs; // 輸入 rs
fake_instr[20:16] = in_ifid_rt; // 輸入 rt

force UUT.IDEX_mem_M = in_idx_M; // 注入 ID/EX 階段的 Memory 控制訊號
force UUT.IDEX_mem_Rt_addr = in_idx_rt; // 注入 ID/EX 階段的 Rt 暫存器位址
force UUT.IFID_mem_Instruction = fake_instr; // 注入 IF/ID 階段的指令內容
#10;
if ((UUT.PC_write != expected_PC_write) ||
    (UUT.IFID_mem_stall != expected_IFID_mem_stall) ||
    (UUT.Control_stall != expected_Control_stall)) begin
    $display("[ERROR] Case %0d", case_id);
```

```
    // release 掉之後, 才可以下新的 force
    release UUT.IDEX_mem_M;
    release UUT.IDEX_mem_Rt_addr;
    release UUT.IFID_mem_Instruction;

    #10;
end
endtask
```

開始執行迴圈

```

$display("\n===== Hazard Detection Unit Test =====");

pattern_file = $fopen(`PATTERN_FILE, "r");
if (pattern_file == 0) begin
    $display("[ERROR] Cannot open pattern file: %s", `PATTERN_FILE);
    $finish;
end

@(negedge clk);
rst_n = 1'b0;
repeat (2) @(negedge clk);
rst_n = 1'b1;
@(negedge clk);
for (i = 0; i < `PATTERN_NUM; i = i + 1) begin
    scan_count = $fscanf(pattern_file, "%h %h %h %h %h %h %h %h\n",
        temp_idx_M,
        temp_idx_rt,
        temp_ifid_rs,
        temp_ifid_rt,
        temp_exp_PC_write,
        temp_exp_IFID_mem_stall,
        temp_exp_Control_stall,
        description_id
    );

```

```

if (scan_count != 8) begin
    $display("[ERROR] Pattern format error at case %0d", i + 1);
    $display("        Expected 8 values, got %0d", scan_count);
    $finish;
end

index_M          = temp_idx_M[2:0];
index_rt         = temp_idx_rt[4:0];
ifid_rs          = temp_ifid_rs[4:0];
ifid_rt          = temp_ifid_rt[4:0];
exp_PC_write     = temp_exp_PC_write[0];
exp_IFID_mem_stall = temp_exp_IFID_mem_stall[0];
exp_Control_stall = temp_exp_Control_stall[0];

run_hazard_pattern(
    index_M,
    index_rt,
    ifid_rs,
    ifid_rt,
    exp_PC_write,
    exp_IFID_mem_stall,
    exp_Control_stall,
    description_id

```

input pattern

1	00000000	00000005	00000005	00000006	00000001	00000001	00000000	00000001
2	00000004	00000005	00000005	00000006	00000000	00000000	00000001	00000002
3	00000004	00000008	00000003	00000008	00000000	00000000	00000001	00000003
4	00000004	0000000a	00000001	00000002	00000001	00000001	00000000	00000004
5	00000002	0000000c	0000000c	00000000	00000001	00000001	00000000	00000005
6	00000004	0000000f	0000000f	0000000f	00000000	00000000	00000001	00000006
7	00000004	00000000	00000000	00000005	00000000	00000000	00000001	00000007
8	00000004	00000012	00000011	00000012	00000000	00000000	00000001	00000008

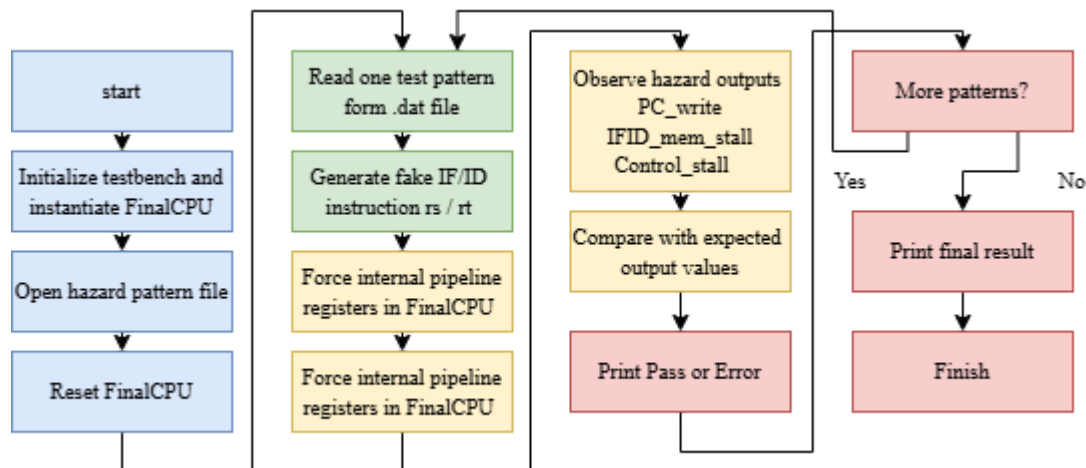
Vivado tcl console

```
===== Hazard Detection Unit Test =====
[PASS ] Case 1
[PASS ] Case 2
[PASS ] Case 3
[PASS ] Case 4
[PASS ] Case 5
[PASS ] Case 6
[PASS ] Case 7
[PASS ] Case 8

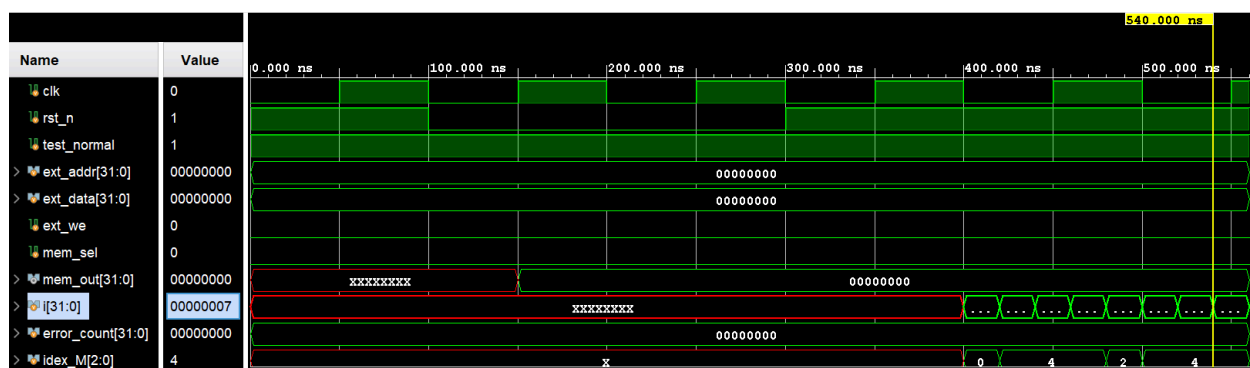
===== Hazard Detection Test Result =====
All Hazard Detection tests PASSED.
$finish called at time : 560 ns : File "C:/Vivados_Verilog_project/PA3/FinalCPU/FinalCPU.srscs/sim_1/new/tk
relaunch_sim: Time (s): cpu = 00:00:00 ; elapsed = 00:00:07 . Memory (MB): peak = 4174.492 ; gain = 0.000
```

輸入的測資全部通過

Flow Chart



Simulation Waveform



可以透過 waveform 看到, testbench 先將 rst_n 拉低兩個 clock cycle 進行 reset, 接著將 rst_n 拉回 1, 等待下一個負緣並開始讀取 Hazard_testpattern.dat 中的測資。由於這個 testbench 是直接利用 force 將測試值注入 FinalCPU 內部訊號, 因此不需要等待完整 pipeline 執行, 只需要等待 combinational logic 穩定後即可比較結果。圖中 i 代表目前執行到的測資 case。

Describe

這段 testbench 是用來驗證 FinalCPU 內部的 hazard detection logic 是否能正確判斷 hazard 條件並且執行 bubble。

Testbench 會從 Hazard_testpattern.dat 讀入測資，每一行代表一組 case。每組資料包含 ID/EX 階段的 memory control signal、ID/EX 的 rt address、IF/ID instruction 的 rs/rt address，以及預期的 PC_write、IFID_mem_stall 和 Control_stall。

由於 Hazard Detection Unit 是直接寫在 FinalCPU 內部，因此 testbench 會 instance FinalCPU，但不執行完整 CPU 流程。測試時會根據 pattern 產生 fake instruction，並用 force 將 idex_M、idex_rt 和 fake instruction 注入 FinalCPU 內部訊號。

注入後，testbench 會等待 combinational logic 穩定，再比較實際輸出的 PC_write、IFID_mem_stall、Control_stall 是否符合 expected value。若相同則輸出 PASS，否則輸出 ERROR 並顯示錯誤訊號。

Input pattern 採用一行一組的形式，每行共有 8 個欄位，分別代表 idex_M, idex_rt, ifid_rs, ifid_rt, expected_PC_write, expected_IFID_mem_stall, expected_Control_stall, case_id

Conclusion

這次作業實作了一個 32-bit 五階段 pipeline CPU，並分成三個部分逐步完成。Part 1 先實作支援 R-format instruction 的基本 pipeline 架構，確認 IF、ID、EX、MEM、WB 五個階段可以正確傳遞資料與控制訊號。Part 2 在此基礎上加入 I-format instruction。Part 3 進一步加入 forwarding unit 與 hazard detection unit，用來解決 data hazard。一般資料相依性問題可透過 forwarding 處理，而 lw-use hazard 則透過 stall PC、暫停 IF/ID register，並插入 bubble 解決。透過 RTL simulation、post-synthesis simulation 與額外 testbench 驗證後，可以確認整體 pipeline CPU 功能正確。

心得

這次的作業相當複雜，甚至是沒有 branch, jump 的情況下。但我發現因為各個模組內部功能其實大致相同，因此只要了解各個 module 之間是如何連接，並且要如何判斷便會變得容易。這次的實作使我實際操作了課堂上所學，使我受益良多，若未來有機會我也會嘗試挑戰加上 branch 以及 jump 的 Pipeline CPU 架構。